

Detailed Description of the OAS Nonlinear Solver

Code inspection note

May 8, 2026

Abstract

This document describes the nonlinear solution algorithm implemented in the OAS solver, with emphasis on the steady-state nonlinear solver path used by `SteadyStateNonLinearSolver`. It follows the code from the model-level time/load-step loop through the Newton-like nonlinear iterations, force assembly, constraint reduction, linear solves, convergence checks, adaptive time step control, restarts, and final state commit. The goal is to make the implementation readable from the solver variables themselves, so most formulas are written next to the corresponding C++ member names.

Contents

1	Scope and Source Map	4
2	High-Level Execution Model	4
3	Notation and Solver Variables	5
3.1	Global Time and Counters	5
3.2	Unknowns, Corrections, and Forces	6
3.3	Matrices	6
4	Initialization	7
4.1	Model Initialization	7
4.2	Base Solver Initialization	8
4.3	Linear and Nonlinear Solver Initialization	8
5	Step Preparation	8
6	Nonlinear Solver Algorithm	9
6.1	Direct-Control Path	9
6.2	Iteration Start: Matrix Update	9
6.3	Residual Reduction	10
6.4	Linear Correction	11
6.5	Trial-State Update	11
6.6	Force Re-Evaluation	11
7	Force Assembly and Residual Construction	11
7.1	Internal and External Force Routine	11
7.2	Element Internal Force Integration	12
7.3	External Force and Reaction Completion	12

8	Boundary Conditions, Constraints, and DoF Mapping	13
8.1	Degree-of-Freedom Numbering	13
8.2	Boundary Updates	13
8.3	Reduced and Full-Space Operations	13
9	Convergence Criteria	14
9.1	Physical-Field Separation	14
9.2	Residual Error	14
9.3	Displacement or Increment Error	14
9.4	Energy Error	15
9.5	Convergence Decision	15
10	Adaptive Linear Tolerance	15
11	Indirect Displacement Control	16
12	Restart, Relaxed Acceptance, and Termination	17
12.1	Normal Convergence	17
12.2	Critical Restart	17
12.3	Relaxed-Tolerance Acceptance	18
12.4	Failure	18
13	Adaptive Time-Step Control	18
14	Step Commit	18
15	Energy Accounting	19
16	Matrix Assembly and Effective System	19
16.1	Element Stiffness Assembly	19
16.2	Effective Matrix Construction	19
17	Linear Solver Interface	20
17.1	Supported Solver Families	20
17.2	Profiling Wrapper	20
18	Runtime Profiling Names	21
19	Reset Path	21
20	Configuration Summary	22
21	What a Single Direct-Control Iteration Means Physically	22
22	Important Implementation Consequences	23
22.1	The Committed State Does Not Move During Iterations	23
22.2	Material Statuses Have Trial and Committed Roles	23
22.3	Prescribed Degrees of Freedom Are Balanced by Reactions	23
22.4	Matrix Update Frequency Changes the Nonlinear Method	23
22.5	Linear Solver Accuracy Can Affect Nonlinear Acceptance	24
23	Compact Algorithm Reference	24
24	Reading Solver Output	25

1 Scope and Source Map

The nonlinear implementation is spread across the model, solver, node, element, and linear-algebra layers. Table 1 lists the files that are most relevant for understanding the control flow.

Table 1: Main source files for the nonlinear solver.

File	Role
<code>src/solver/src/model.cpp</code>	Owns the global analysis loop in <code>Model::solve()</code> . It calls <code>solver->solveStep()</code> , exports each accepted step, and reports step wall time.
<code>src/solver/src/solver.h</code>	Defines the base <code>Solver</code> state: time variables, displacement vectors, forces, residuals, stiffness matrices, tolerances, energies, and the common <code>solveStep()</code> wrapper.
<code>src/solver/src/solver.cpp</code>	Implements base initialization, force computation, energy accumulation, step preparation, step commit, perturbations, and termination handling.
<code>src/solver/src/solver_implicit.h</code>	Declares <code>SteadyStateLinearSolver</code> and <code>SteadyStateNonLinearSolver</code> . This header also contains the nonlinear control parameters, default tolerances, matrix-update settings, adaptive linear-solve settings, and indirect displacement control state.
<code>src/solver/src/solver_implicit.cpp</code>	Implements the nonlinear solve loop, convergence checks, restart logic, matrix-update policy, effective matrix construction, linear solver creation, profiling wrappers, reset path, and adaptive linear tolerance logic.
<code>src/solver/src/node_container.cpp</code>	Implements full-to-reduced DoF mapping, prescribed boundary updates, constraint handling, reduced force extraction, external reaction update, and simplex volumetric strain update.
<code>src/solver/src/element_container.cpp</code>	Implements internal force integration, element strain/stress evaluation, material status updates, structural matrix pattern preparation, and stiffness matrix assembly.
<code>src/solver/src/element.cpp</code> and <code>src/solver/src/element.h</code>	Define element-level strain, stress, internal force, and stiffness contributions. Specialized element types can override parts of this behavior.
<code>src/solver/src/linalg.cpp</code> and <code>src/solver/src/linalg_profile.*</code>	Define the direct and iterative linear solver wrappers, including profiling events used by the nonlinear loop.

The discussion below focuses on `SteadyStateNonLinearSolver`. Transient mechanical solvers inherit the same general structure but override parts of the force and energy evaluation to include damping, inertia, and kinetic energy.

2 High-Level Execution Model

At the top level, OAS advances through accepted load/time steps. Each step is solved by a nonlinear iteration loop. In the ordinary direct-control path, one linearized correction is computed per nonlinear iteration:

$$K_{\text{eff}} \Delta r_f = R_f,$$

where R_f is the reduced residual on free degrees of freedom and Δr_f is the reduced correction. The correction is expanded into the full degree-of-freedom space, constraints are applied, the trial unknown vector is updated, new internal and external forces are evaluated, and convergence errors are checked.

The model-level loop is:

```
export initial state
while solver is not terminated:
    solver.solveStep()
    export accepted step
    print step timing
```

The solver-level `solveStep()` wrapper is common to all solvers:

```
runBeforeEachStep()
solve()
runAfterEachStep()
```

For the nonlinear solver, `solve()` may restart the same step with a smaller time increment if convergence fails. Only after `solve()` returns does `runAfterEachStep()` commit the accepted state.

3 Notation and Solver Variables

3.1 Global Time and Counters

Table 2: Time-step and iteration notation.

Symbol	C++ member or local	Meaning
n	<code>step</code>	Accepted load/time-step index. It is incremented in <code>Solver::runBeforeEachStep()</code> .
t_n	<code>time</code>	Target time/load coordinate of the current step. The solver may snap this to important boundary-condition or function extrema.
Δt_n	<code>dt</code>	Current time/load increment. It can be enlarged, reduced, or critically reduced after failed convergence.
$\Delta t_{\min}, \Delta t_{\max}$	<code>dtmin, dtmax</code>	Lower and upper bounds used by adaptive time stepping.
i	<code>it</code>	Nonlinear iteration counter inside the current step. It starts at zero after each restart.
i_{cum}	<code>cumul_it</code>	Cumulative nonlinear iteration counter within the current call to <code>solve()</code> . It does not reset after a normal iteration, and it is used by the cumulative stiffness-update rule.
$N_{\max}$	<code>maxIt</code>	Maximum allowed nonlinear iterations before restart or failure.
$N_{\min}$	<code>minIt</code>	Minimum nonlinear iterations required before a converged step can be accepted.
N_{enlarge}	<code>enlargeIt</code>	If convergence is reached before this iteration count, the next step may be enlarged.

Symbol	C++ member or local	Meaning
N_{shorten}	<code>shortenIt</code>	If convergence is reached after this iteration count, the next step may be shortened.

3.2 Unknowns, Corrections, and Forces

The code distinguishes full vectors, which contain all nodal degrees of freedom, from reduced vectors, which contain only free degrees of freedom. Constrained or prescribed degrees of freedom are recovered through the node and constraint containers.

Table 3: Primary nonlinear vectors.

Symbol	C++ member	Meaning
r_n	<code>r</code>	Committed full solution vector from the last accepted step. It is updated only after the current step is accepted.
r^{tr}	<code>trial_r</code>	Current trial full solution vector inside the nonlinear loop.
Δr_f	<code>ddr</code>	Reduced correction on free degrees of freedom, returned by the linear solver.
Δr	<code>full_ddr</code>	Full-space correction obtained by expanding <code>ddr</code> and applying dependency relations.
f_{int}	<code>f_int</code>	Full internal force vector assembled from element stresses and internal sources.
f_{ext}	<code>f_ext</code>	Full external force vector after nodal loads, prescribed-reaction updates, and constraint reaction contributions are applied.
f_{load}	<code>load</code>	Full prescribed load vector assembled from nodal and body loads before reaction completion.
f_{dam}	<code>f_dam</code>	Damping force vector. In the steady-state nonlinear path this remains zero; transient solvers can add damping contributions.
f_{acc}	<code>f_acc</code>	Inertial force vector. In the steady-state nonlinear path this remains zero; transient solvers can add acceleration contributions.
R	<code>residuals</code>	Full residual vector. In the steady-state nonlinear path, $R = f_{\text{ext}} - f_{\text{int}}$.
R_f	<code>f</code>	Reduced right-hand side extracted from <code>residuals</code> for free degrees of freedom after constraint master-force transfer.
$f_{\text{int},n}, f_{\text{ext},n}$	<code>f_int_old, f_ext_old</code>	Committed force vectors from the last accepted step. They are used for energy integration and restart restoration.

3.3 Matrices

Table 4: Matrix notation.

Symbol	C++ member	Meaning
K	<code>K</code>	Structural tangent, secant, elastic, or consistent matrix assembled from elements before the final effective-system transformation.
K_{eff}	<code>Keff</code>	Effective matrix solved by the linear solver. In the steady-state nonlinear path it starts as K , then constraints are transformed into the active system space. Transient solvers can add damping or mass terms.
K_e	element local matrix	Element contribution returned by <code>Element::giveStiffnessMatrix()</code> or related damping/mass methods.
P	implicit node/constraint map	The full-to-reduced and reduced-to-full mapping implemented by <code>NodeContainer</code> and <code>ConstraintContainer</code> . The code does not store a single dense matrix P ; it applies the map procedurally.

4 Initialization

4.1 Model Initialization

`Model::init()` prepares the simulation in a fixed order:

1. Materials are initialized.
2. Elements are initialized and preprocessing blocks are run.
3. Boundary-condition objects are initialized.
4. Nodes are initialized.
5. Simplex connectivity is initialized when the material setup requires simplex volumetric strain support.
6. Constraints and boundary conditions establish their active degree-of- freedom relations.
7. Fiber/material auxiliary assignments are applied.
8. Element neighbor or friend relations are initialized.
9. The solver is initialized.
10. Exporters are initialized.

This order matters because the solver initialization needs a finalized degree- of-freedom numbering, element list, material statuses, and boundary-condition state.

4.2 Base Solver Initialization

`Solver::init()` allocates all primary full and reduced vectors:

$$r, r^{\text{tr}}, f_{\text{int}}, f_{\text{ext}}, f_{\text{dam}}, f_{\text{acc}}, R \in \mathbb{R}^{N_{\text{dof}}},$$

and

$$\Delta r_f, R_f \in \mathbb{R}^{N_{\text{free}}}.$$

Initial conditions are read if an initial-condition file is configured; otherwise the state is set to zero. The trial vector is initialized from the committed vector:

$$r^{\text{tr}} \leftarrow r.$$

The solver then computes initial internal and external forces:

$$\text{computeInternalExternalForces}(r, f_{\text{load}}, \text{frozen}=\text{false}, 0, -1).$$

Finally, element material statuses are committed through `elems->updateMaterialStatuses()`.

4.3 Linear and Nonlinear Solver Initialization

`SteadyStateLinearSolver::init()` extends the base initialization by:

1. Initializing runtime and linear-solver profilers.
2. Preparing the sparsity pattern through `elems->prepareStiffnessMatrix(K)`.
3. Calling `updateSystemMatrices(0, 0, true)` to assemble the first structural matrix.
4. Calling `computeKeff()` to build, transform, and factorize the first effective system matrix.

`SteadyStateNonLinearSolver::init()` additionally:

1. Loads material-status restart data if requested.
2. Initializes nonlinear convergence state.
3. Initializes indirect displacement control if configured.
4. Evaluates forces at the initial integration time with frozen variables to initialize field quantities consistently.

5 Step Preparation

Every accepted-step attempt starts with `runBeforeEachStep()`. The base implementation performs the following operations:

1. Increment the step counter:

$$n \leftarrow n + 1.$$

2. Choose the next target time with `setNextStepTime()`.

3. Store the old load vector:

$$f_{\text{load},n} \leftarrow f_{\text{load}}.$$

4. Reset the current load vector and reduced correction:

$$f_{\text{load}} \leftarrow 0, \quad \Delta r_f \leftarrow 0.$$

`setNextStepTime()` normally advances t by Δt , but it also looks ahead to the next important time from time-dependent functions and boundary conditions. If such a point lies within $t+1.25\Delta t$, the step target snaps to that exact point. If the point is a boundary-condition stage transition, the model jumps to the next boundary-condition stage.

`SteadyStateLinearSolver::runBeforeEachStep()` sets:

$$r^{\text{tr}} \leftarrow r,$$

and prints the step header. `SteadyStateNonLinearSolver::runBeforeEachStep()` adds initialization for indirect displacement control and prints the nonlinear error tolerance header.

6 Nonlinear Solver Algorithm

6.1 Direct-Control Path

The ordinary nonlinear path is used when no indirect displacement control object is configured. The structure of `SteadyStateNonLinearSolver::solve()` is:

```

converged = false
restarted = false
while not converged:
    assemble load and boundary conditions for current time
    compute frozen forces at integration time
    it = 0
    while not converged and it < maxIt:
        maybe update stiffness matrix and factorize Keff
        reduce full residual R to reduced RHS R_f
        solve Keff * ddr = R_f
        expand ddr to full_ddr and update trial_r
        compute unfrozen forces at updated trial_r
        evaluate residual, displacement, and energy errors
        decide whether convergence is acceptable
        export iteration data
        it += 1
    if converged:
        compute step-end forces
    else:
        restart, relax tolerance, or terminate
        adapt next dt if accepted

```

The nonlinear iteration is Newton-like: it repeatedly solves a linearized problem, updates the trial state, and checks whether residual, increment, and energy measures are small enough. The exact meaning of the matrix depends on the configured stiffness matrix type and update frequency. With a tangent matrix and frequent updates the method is closest to Newton's method. With secant or less frequent updates it becomes a modified Newton or quasi-Newton procedure.

6.2 Iteration Start: Matrix Update

At the beginning of every nonlinear iteration, the solver may update the structural matrix:

$$K \leftarrow \sum_e K_e,$$

then build and factorize

$$K_{\text{eff}}.$$

The update is controlled by `updateSystemMatrices(iteration, cumul_iteration, enforce)`. The matrix is updated if at least one of the following conditions is true:

- `enforce` is true.
- The current step/iteration satisfies the configured step-update rule.
- `stiffnessMatrixIterUpdate==0`, meaning every iteration.
- A positive iteration-period rule matches the current iteration.
- A positive cumulative-iteration-period rule matches the cumulative counter.

When the matrix is updated, the first nonlinear iteration can use a special first-iteration matrix type if `stiffMatTypeFirstIT` is not `void`. Otherwise it uses `stiffMatType`. Valid matrix types are shown in Table 5.

Table 5: Configured stiffness matrix types.

Type	Meaning in the code
<code>elastic</code>	Uses an elastic stiffness contribution from material statuses. This is usually more robust but less nonlinear-state-specific.
<code>secant</code>	Uses a secant material stiffness. This is the default in the nonlinear solver.
<code>tangent</code>	Uses a tangent stiffness, when supplied by the material model. This is the closest option to a classical Newton tangent.
<code>consistent</code>	Uses a consistent material stiffness contribution, when supplied.

After assembly, `computeKeff()` copies:

$$K_{\text{eff}} \leftarrow K,$$

then transforms the matrix into constraint space if constraints are active. Finally, `factorizeLinearSystem()` analyzes or factorizes the selected linear solver backend.

6.3 Residual Reduction

The force vector used by the linear solver is not the raw full residual. The solver calls:

```
nodes->giveReducedForceArray(residuals, f).
```

This first transfers slave or constrained forces back to their master degrees of freedom:

$$R \leftarrow \text{constraint-master-force-transform}(R),$$

then extracts only the free active degrees of freedom:

$$R_f = R|_{\text{free}}.$$

This reduced vector is the right-hand side of the linear solve.

6.4 Linear Correction

The direct-control correction is:

$$K_{\text{eff}}\Delta r_f = R_f.$$

In code this is:

```
profiledLinearSolve(DDR, f, "direct_residual").
```

The wrapper records timing and linear-solver metadata. For iterative solvers it also records iteration counts, requested tolerance, achieved error, deflation statistics, and specialized sub-phase timings if available.

If the solve fails, the nonlinear step is terminated immediately. If the solve succeeds and the solver supports deflation vectors, the correction from a successful direct residual solve can be collected:

```
collectLinearDeflationVector(DDR, "direct_residual", success).
```

6.5 Trial-State Update

The reduced correction is expanded into the full degree-of-freedom space:

$$\Delta r \leftarrow P\Delta r_f,$$

through:

```
nodes->giveFullDoFArray(DDR, full_DDR).
```

The node and constraint layer then accounts for dependency relations that depend on conjugate variables:

```
nodes->updateFullDoFsByDependenciesOnConjugates(full_DDR, trial_r, f_ext).
```

Finally:

$$r^{\text{tr}} \leftarrow r^{\text{tr}} + \Delta r.$$

6.6 Force Re-Evaluation

After the trial vector is updated, the solver recomputes forces at the new trial state:

```
computeForcesAtIntegrationTime(false).
```

For the steady-state nonlinear solver this calls:

```
computeInternalExternalForces(trial_r, load, frozen=false, time, it).
```

The resulting steady residual is:

$$R = f_{\text{ext}} - f_{\text{int}}.$$

7 Force Assembly and Residual Construction

7.1 Internal and External Force Routine

`Solver::computeInternalExternalForces()` performs three main operations:

1. Update simplex volumetric strains:

```
nodes->updateSimplexVolumetricStrains(rr).
```

2. Integrate element internal forces:

```
elems->integrateInternalForces(rr, f_int, frozen, t, timeStep).
```

3. Complete external forces and reactions:

```
nodes->updateExternalForcesByReactions(...).
```

The steady residual is then formed explicitly:

$$R = f_{\text{ext}} - f_{\text{int}}.$$

7.2 Element Internal Force Integration

`ElementContainer::integrateInternalForces()` assembles the full internal force vector. Its conceptual algorithm is:

```
f_int = 0
reset eigenstrain contributions
run material preparation for stress evaluation
for solution_order = 0 ... max_solution_order:
    for each element with this solution_order:
        gather element dof values from trial_r
        evaluate element strains
        evaluate element stresses
for each element:
    compute element internal force vector
    scatter local force entries into global f_int
```

The `solution_order` loop is important for coupled or dependent element quantities. It enforces a deterministic order between element groups whose stress/strain evaluation may depend on previously updated fields.

The `frozen` argument controls material-state handling:

- **frozen=true**: stresses are evaluated with frozen internal variables. This is used for predictor-style or initialization passes where the code wants forces without advancing temporary material state.
- **frozen=false**: stresses are evaluated normally for the current trial state, updating temporary material-status quantities used by the current nonlinear iteration.

7.3 External Force and Reaction Completion

The initial `load` vector contains applied nodal and body-load terms. It is not yet the final `f_ext`. The node container completes the external force vector by:

1. Transforming internal, damping, acceleration, and load vectors through constraint master-force relations.
2. Starting from:

$$f_{\text{ext}} \leftarrow f_{\text{load}}.$$

3. Adding Lagrange-multiplier reaction terms to master degrees of freedom.
4. For directly prescribed blocked degrees of freedom, setting the external force equal to the force required for equilibrium:

$$f_{\text{ext},i} = f_{\text{int},i} + f_{\text{dam},i} + f_{\text{acc},i}.$$

This is why the full residual should vanish on prescribed degrees of freedom: their reaction force is defined from the internal, damping, and inertial force balance.

8 Boundary Conditions, Constraints, and DoF Mapping

8.1 Degree-of-Freedom Numbering

`NodeContainer::establishDoFArray()` builds the solver numbering. The layout is:

1. Free degrees of freedom.
2. Slave constrained degrees of freedom.
3. Directly blocked or prescribed degrees of freedom.

Only the first group is part of the reduced linear system. The inverse map `invDoFid` maps reduced indices back to full degree-of-freedom indices. The physical field of every degree of freedom is stored in `physicalFieldsDoF`; this is later used by the convergence-error calculation.

8.2 Boundary Updates

For direct control, at the start of an attempted step:

```
nodes->addRHS_nodalLoad(load, time)
```

adds active body and nodal loads at the current time. Then:

```
nodes->updateDirichletBC(trial_r, time)
```

writes prescribed displacement or field values into the full trial vector and updates dependent constrained degrees of freedom.

8.3 Reduced and Full-Space Operations

The most important map operations are:

Table 6: Node and constraint mapping routines used by the nonlinear solver.

Routine	Meaning
<code>giveReducedForceArray(fullf, f)</code>	Transfers constrained slave forces to masters, then extracts free rows into a reduced vector.
<code>giveFullDoFArray(fDoFs, fullDoFs)</code>	Expands a reduced correction into the full vector and calculates dependent constrained degrees of freedom.
<code>updateFullDoFsByDependenciesOnConjugates(.)</code>	Applies additional correction terms for constraints whose dependent values are functions of conjugate variables such as forces.
<code>updateExternalForcesByReactions(...)</code>	Completes <code>f_ext</code> by adding reactions and making prescribed degrees of freedom force-balanced.

9 Convergence Criteria

9.1 Physical-Field Separation

The nonlinear solver computes convergence errors separately for each physical field, then combines them. The current field indices are listed in Table 7.

Table 7: Physical fields used by the error calculation.

Index	Field	Stabilizing floor ϵ_p^2
0	Mechanics	10^{-20}
1	Transport	10^{-10}
2	Temperature	10^{-17}
3	Humidity	10^{-18}

Let I_p be the set of full degrees of freedom belonging to physical field p . The code accumulates squared norms:

$$S_R^p = \sum_{i \in I_p} R_i^2, \quad S_{\Delta r}^p = \sum_{i \in I_p} (\Delta r_i)^2, \quad S_r^p = \sum_{i \in I_p} (r_i^{\text{tr}})^2.$$

It also accumulates force scales:

$$S_{\text{ext}}^p = \sum_{i \in I_p} f_{\text{ext},i}^2, \quad S_{\text{int}}^p = \sum_{i \in I_p} f_{\text{int},i}^2, \quad S_{\text{dam}}^p = \sum_{i \in I_p} f_{\text{dam},i}^2, \quad S_{\text{acc}}^p = \sum_{i \in I_p} f_{\text{acc},i}^2.$$

For energy, it accumulates:

$$E_R^p = \sum_{i \in I_p} R_i \Delta r_i.$$

Some elements or RVE/eigenstrain mechanisms can add external error contributions to the displacement, force, and external-work scales. The code adds these before final normalization.

9.2 Residual Error

The residual error is:

$$e_R = \sqrt{\sum_p \frac{S_R^p}{\max(S_{\text{ext}}^p, S_{\text{int}}^p, S_{\text{dam}}^p, S_{\text{acc}}^p, \epsilon_p^2)}}.$$

In code this value is stored in **resErr**. It measures force imbalance relative to the largest relevant force scale in each physical field.

9.3 Displacement or Increment Error

The displacement increment error is:

$$e_{\Delta r} = \sqrt{\sum_p \frac{S_{\Delta r}^p}{\max(S_r^p, \epsilon_p^2)}}.$$

In code this value is stored in **disErr**. On the first nonlinear iteration of a step, the code sets the displayed and tested displacement error to zero after evaluation:

$$i = 0 \quad \Rightarrow \quad e_{\Delta r} \leftarrow 0.$$

This avoids rejecting the first correction based only on an increment compared with a still-developing trial scale.

9.4 Energy Error

The energy error is:

$$e_E = \sqrt{\sum_p \frac{|E_R^p|}{\max(|W_{\text{ext}}^p|, |W_{\text{int}}^p|, |W_{\text{kin}}^p|, \epsilon_p^2)}}.$$

In code this value is stored in `eneErr`. It measures residual work during the latest correction relative to accumulated external, internal, or kinetic energy scales.

9.5 Convergence Decision

The basic convergence test is:

$$e_{\Delta r} \leq \tau_{\Delta r}, \quad e_R \leq \tau_R, \quad e_E \leq \tau_E, \quad i + 1 \geq N_{\min}.$$

The tolerances are:

Table 8: Nonlinear tolerances.

Quantity	C++ member	Default
$\tau_{\Delta r}$	<code>maxDisErr</code>	10^{-5}
τ_R	<code>maxResErr</code>	10^{-5}
τ_E	<code>maxEneErr</code>	10^{-5}
$\tau_{\Delta r}^{\text{limit}}$	<code>limitDisErr</code>	0 unless configured
τ_R^{limit}	<code>limitResErr</code>	0 unless configured
τ_E^{limit}	<code>limitEneErr</code>	0 unless configured

If adaptive linear tolerance is active, the nonlinear solver can require one additional correction with a tighter linear tolerance before accepting a step that otherwise satisfies the nonlinear errors. This prevents a loose linear solve from being the last correction in a nominally converged nonlinear step when strict final convergence is requested.

If any of the nonlinear errors is NaN, the current attempt is treated as failed. The iteration counter is forced to the maximum and a large residual error is assigned.

10 Adaptive Linear Tolerance

The nonlinear solver can dynamically switch the requested tolerance of the linear solver during direct residual solves. This is controlled by the adaptive-linear-tolerance members, especially:

- `linearAdaptiveTolEnabled`;
- `linearAdaptiveLooseTol`;
- `linearAdaptiveTightTol`;
- `linearAdaptiveTriggerRatio`;
- `linearAdaptiveRequireTightConvergence`.

The rule is:

1. Adaptive tolerance only applies to RHS kind `direct_residual`.
2. The loose tolerance is used by default.

3. The tight tolerance is used if the solver has requested a strict final solve or if the current nonlinear error ratio is already below the configured trigger ratio.
4. The nonlinear error ratio is the maximum of the residual, energy, and displacement errors divided by their target tolerances. The displacement component is ignored for the first nonlinear iteration.
5. If a step appears converged after a loose linear solve and strict final convergence is required, the solver performs one more iteration using the tight linear tolerance.

This mechanism is meant to avoid oversolving early nonlinear iterations while still ending each accepted nonlinear step with a sufficiently accurate linear correction.

11 Indirect Displacement Control

When `idc` is configured, the nonlinear solver does not simply solve one linear correction per iteration. It solves two related systems and computes a load multiplier correction.

The variables are:

Table 9: Indirect displacement control variables.

Symbol	C++ member	Meaning
λ	<code>idc_time</code>	The indirect-control load coordinate currently used to evaluate loads and boundary conditions.
λ_n	<code>idc_time_converged</code>	Last accepted indirect-control load coordinate.
$\Delta\lambda_0$	<code>idc_dt</code>	Small probe increment used to form a load-increment direction.
μ	<code>load_mult</code>	Multiplier correction returned by the indirect-control object.
Δr_f^R	<code>ddr</code>	Residual correction direction.
Δr_f^L	<code>ddf</code>	Load-increment correction direction.

The conceptual sequence is:

1. Save the current trial state and reduced residual.

2. Solve:

$$K_{\text{eff}}\Delta r_f^R = R_f.$$

3. Advance the indirect-control load coordinate by $\Delta\lambda_0$, update loads and boundary conditions, and recompute forces.

4. Solve:

$$K_{\text{eff}}\Delta r_f^L = R_f(\lambda + \Delta\lambda_0) - R_f(\lambda).$$

5. Form trial states A and B, optionally evaluate external forces for both states, and ask the indirect-control object for the multiplier μ .

6. Correct the residual direction:

$$\Delta r_f \leftarrow \Delta r_f^R + \mu\Delta r_f^L.$$

7. Update:

$$\lambda \leftarrow \lambda + \mu \Delta \lambda_0.$$

8. Restore the original trial state, apply the corrected load and boundary conditions at the new indirect-control coordinate, then apply the corrected displacement increment.

On restart, `idc_time` is reset to `idc_time_converged`, so the failed trial load advancement is discarded.

12 Restart, Relaxed Acceptance, and Termination

When the inner nonlinear loop ends, the solver has either converged or exhausted the nonlinear iteration budget.

12.1 Normal Convergence

If the convergence criteria are satisfied, the solver marks:

$$\text{fully_converged} \leftarrow \text{true},$$

and computes step-end forces:

$$\text{computeForcesAtStepEnd}(\text{false}).$$

For the steady-state nonlinear solver, this is another force evaluation at the accepted trial state. It ensures the final force state is consistent before the step is committed.

12.2 Critical Restart

If the step did not converge and the current increment is larger than the minimum increment, the solver restarts the same step:

$$\Delta t \leftarrow \max(\gamma_{\text{crit}} \Delta t, \Delta t_{\text{min}}),$$

where:

$$\gamma_{\text{crit}} = \text{critical_step_decrease}.$$

The default is 0.5.

The restart restores:

- the target time to the previous value, then advances it by the reduced Δt ;
- `trial_r` back to the last committed `r`;
- old internal and external forces;
- the load vector and reduced correction to zero;
- material statuses through `elems->resetMaterialStatuses()`;
- indirect-control state, if active.

The nonlinear loop then tries the same physical step again with the smaller increment.

12.3 Relaxed-Tolerance Acceptance

If the step still did not meet the main tolerances, but the errors satisfy the configured limit tolerances, the solver can accept the step as not fully converged:

`fully_converged` $\leftarrow$ `false`.

This path is only available when nonzero limit tolerances were configured. Otherwise, the default limit tolerances are zero and this fallback is disabled.

12.4 Failure

If the step cannot be restarted and cannot be accepted under relaxed tolerances, the solver prints an error message and sets:

`terminated` $\leftarrow$ `true`.

The model-level solve loop then stops.

13 Adaptive Time-Step Control

After an accepted step, the next increment can be adjusted based on the number of nonlinear iterations:

$$\Delta t_{\text{next}} = \begin{cases} \min(\gamma_{\text{inc}}\Delta t, \Delta t_{\text{max}}), & \text{if } i < N_{\text{enlarge}} \text{ and no restart occurred,} \\ \max(\gamma_{\text{dec}}\Delta t, \Delta t_{\text{min}}), & \text{if } i > N_{\text{shorten}}, \\ \Delta t, & \text{otherwise.} \end{cases}$$

The default multipliers are:

Table 10: Adaptive time-step factors.

Meaning	C++ member	Default
Normal enlargement factor	<code>step_increase</code>	1.25
Normal reduction factor	<code>step_decrease</code>	0.8
Critical restart reduction factor	<code>critical_step_decrease</code>	0.5

The final increment is clamped to $[\Delta t_{\text{min}}, \Delta t_{\text{max}}]$. The base `runAfterEachStep()` also reduces the next increment if it would pass the configured termination time.

14 Step Commit

After `solve()` returns, `solveStep()` calls `runAfterEachStep()`. The base implementation performs the actual state commit:

1. Apply scheduled perturbations if they are active at the current time.
2. Compute total internal, external, and kinetic energies.
3. Commit the trial vector:

$$r \leftarrow r^{\text{tr}}.$$

4. Store old force vectors:

$$f_{\text{int},n} \leftarrow f_{\text{int}}, \quad f_{\text{ext},n} \leftarrow f_{\text{ext}}.$$

5. Commit element material statuses:

$$\text{elems} \rightarrow \text{updateMaterialStatuses}().$$

6. Clamp or terminate based on the next Δt .

7. Commit old energy values.

The nonlinear override prints a separator and, for indirect displacement control, commits:

$$\text{idc_time_converged} \leftarrow \text{idc_time}.$$

15 Energy Accounting

The base energy update integrates internal and external work with a trapezoidal rule over the accepted correction:

$$W_{\text{int}}^p \leftarrow W_{\text{int},n}^p + \sum_{i \in I_p} \frac{1}{2} (f_{\text{int},i} + f_{\text{int},n,i}) (r_i^{\text{tr}} - r_{n,i}),$$

$$W_{\text{ext}}^p \leftarrow W_{\text{ext},n}^p + \sum_{i \in I_p} \frac{1}{2} (f_{\text{ext},i} + f_{\text{ext},n,i}) (r_i^{\text{tr}} - r_{n,i}).$$

In the steady-state base class, kinetic energy is not updated. Transient mechanical solvers can override this behavior to include a meaningful W_{kin} .

16 Matrix Assembly and Effective System

16.1 Element Stiffness Assembly

The element container prepares the sparsity pattern before the nonlinear solve starts. During a matrix update it:

1. Resets matrix values while preserving the pattern.
2. Adds Lagrange multiplier constraint entries.
3. Requests local element matrices K_e from each element.
4. Scatters local entries into global matrix entries using element degree-of-freedom indices.

The code supports optimized assembly paths using a precomputed sparse-entry map when the current matrix pattern is compatible. The mathematical result remains the same: the global structural matrix is the sum of all element and constraint contributions.

16.2 Effective Matrix Construction

For the steady-state nonlinear solver:

$$K_{\text{eff}} \leftarrow K.$$

If constraints exist:

$$K_{\text{eff}} \leftarrow \text{constraint-space-transform}(K_{\text{eff}}).$$

The transformed matrix is then passed to the configured linear solver. The same high-level function also handles linear solver creation on first use and calls symbolic analysis where required.

17 Linear Solver Interface

17.1 Supported Solver Families

`factorizeLinearSystem()` selects the configured solver backend. The available choices depend on compile-time flags. The implemented families include:

- Eigen direct and iterative solvers.
- Pardiso, Cholmod, SuperLU, when compiled in.
- AMGCL conjugate-gradient solver, when compiled in.
- Hypre BoomerAMG conjugate-gradient solver, when compiled in.
- Native DeflatedFGMRES.

For AMGCL, Hypre, and native deflated FGMRES paths, the solver may also build an elastic degree-of-freedom map used for block structure, nullspace hints, lifting, or deflation metadata.

17.2 Profiling Wrapper

The nonlinear solver does not call the linear solver directly. It calls:

```
profiledLinearSolve(x, b, rhsKind).
```

The wrapper records:

- total linear solve time;
- matrix event number and nonlinear iteration context;
- right-hand-side kind;
- right-hand-side norm and change from the previous RHS;
- solution norm;
- linear iteration count and reported error;
- requested tolerance;
- deflation basis size and deflation update metrics, when available;
- detailed sub-phase timings for native deflated FGMRES, when available.

Table 11: RHS kinds used by the nonlinear and reset paths.

RHS kind	Meaning
<code>direct_residual</code>	Ordinary nonlinear correction for direct-control steps. Successful solves can contribute vectors to the deflation basis.
<code>idc_residual</code>	Residual correction direction for indirect displacement control.
<code>idc_load_increment</code>	Load-increment correction direction for indirect displacement control.
<code>reset_residual</code>	Residual correction used by the nonlinear reset procedure.

RHS kind	Meaning
<code>linear_step</code>	Single linear solve used by the steady-state linear solver path.

18 Runtime Profiling Names

The solver contains nested timing scopes. The exact set can vary with compile options and selected solver, but the nonlinear path commonly records the categories shown in Table 12.

Table 12: Important runtime profile names.

Profile name	Meaning
<code>model.solve.solve_step</code>	Wall time spent in one call to <code>solver->solveStep()</code> .
<code>solver.before_step</code>	Step preparation, trial-state reset, time update, and nonlinear before-step logic.
<code>solver.solve</code>	Main solver body for the current step.
<code>solver.after_step</code>	Accepted-step commit, energy update, material status commit, and termination checks.
<code>nonlinear.iteration</code>	One nonlinear iteration, including matrix update when triggered, linear solve, state update, force evaluation, and error evaluation.
<code>matrix.stiffness_update</code>	Structural stiffness matrix assembly.
<code>matrix.compute_keff</code>	Effective matrix construction, constraint transform, and factorization.
<code>linear.solve_total</code>	One profiled linear solve.
<code>forces.total</code>	Total force evaluation wrapper.
<code>forces.simplex_volumetric_strains</code>	Simplex volumetric strain update before element force integration.
<code>forces.integrate_internal_forces</code>	Element strain, stress, and internal-force integration.
<code>forces.external_reactions</code>	Completion of external forces and reactions after internal force assembly.
<code>nonlinear.evaluate_errors</code>	Residual, displacement, and energy error calculation.

Native deflated FGMRES can further split `linear.solve_total` into matvec, preconditioner setup/apply, orthogonalization, reorthogonalization, deflation projection, least-squares work, and residual checks.

19 Reset Path

`SteadyStateNonLinearSolver::reset()` is a separate nonlinear correction path used when the model state must be re-equilibrated without advancing the usual load step. It follows the same basic pattern:

1. Compute forces with frozen internal variables.
2. Repeatedly update the matrix as needed.
3. Reduce the residual.
4. Solve `reset_residual`.

5. Update the trial vector.
6. Recompute frozen forces.
7. Evaluate convergence errors.

Unlike the normal `solve()` method, the reset routine calls `runAfterEachStep()` itself after it converges. It is therefore a self-contained equilibration and commit operation.

20 Configuration Summary

Table 13: Important nonlinear configuration options read by the solver.

Input concept	C++ member	Effect
Initial time step	<code>initdt, dt</code>	Starting load/time increment.
Minimum/maximum time step	<code>dtmin, dtmax</code>	Enable and bound adaptive time stepping.
Residual tolerance	<code>maxResErr</code>	Target residual error.
Energy tolerance	<code>maxEneErr</code>	Target residual-work error.
Increment tolerance	<code>maxDisErr</code>	Target correction-size error.
Limit tolerances	<code>limitResErr, limitEneErr, limitDisErr</code>	Optional relaxed acceptance thresholds.
Minimum/maximum nonlinear iterations	<code>minIt, maxIt</code>	Lower and upper nonlinear iteration limits.
Step enlarge/shorten iteration counts	<code>enlargeIt, shortenIt</code>	Iteration thresholds for next-step adaptation.
Step increase/decrease factors	<code>step_increase, step_decrease</code>	Normal accepted-step time-step scaling.
Critical decrease factor	<code>critical_step_decrease</code>	Failed-step restart scaling.
Matrix update by step	<code>stiffnessMatrixStepUpdate</code>	Controls whether the first iteration of some steps updates the matrix.
Matrix update by iteration	<code>stiffnessMatrixIterUpdate</code>	Controls per-step nonlinear iteration update frequency.
Matrix update by cumulative iteration	<code>stiffnessMatrixCumulIterUpdate</code>	Controls matrix update frequency across restarts or cumulative iterations.
Matrix type	<code>stiffMatType</code>	Selects <code>elastic</code> , <code>secant</code> , <code>tangent</code> , or <code>consistent</code> .
First-iteration matrix type	<code>stiffMatTypeFirstIT</code>	Optional different matrix type for the first nonlinear iteration.
Adaptive linear tolerance	<code>linearAdaptiveTolEnabled</code> and related members	Switches iterative linear solve tolerance between loose and tight values inside the nonlinear process.

21 What a Single Direct-Control Iteration Means Physically

For mechanics, the nonlinear iteration can be read as the following equilibrium correction:

1. Given the current trial displacement u^{tr} , compute the internal nodal force $f_{\text{int}}(u^{\text{tr}})$ from element stresses.
2. Compute the external nodal force f_{ext} , including reactions for prescribed degrees of freedom.
3. The imbalance is

$$R = f_{\text{ext}} - f_{\text{int}}.$$

4. Reduce that imbalance to the active free system R_f .
5. Solve a linearized equilibrium correction

$$K_{\text{eff}} \Delta u_f = R_f.$$

6. Expand Δu_f , update constrained dependent quantities, and set

$$u^{\text{tr}} \leftarrow u^{\text{tr}} + \Delta u.$$

7. Repeat until the remaining force imbalance, correction size, and residual work are all small relative to their natural scales.

For transport, temperature, and humidity fields the same algebraic structure is used. The physical meaning of r , f_{int} , and f_{ext} changes with the field, and the convergence calculation normalizes each field separately before combining them.

22 Important Implementation Consequences

22.1 The Committed State Does Not Move During Iterations

During nonlinear iterations:

$$r \quad \text{remains fixed,} \quad r^{\text{tr}} \quad \text{changes.}$$

This distinction is important for restarts. A failed step can restore $r^{\text{tr}} \leftarrow r$ without losing the last accepted state.

22.2 Material Statuses Have Trial and Committed Roles

Element force evaluation can update temporary material-status quantities during the trial process. These are only committed by `elems->updateMaterialStatuses()` after the step is accepted. If a step is restarted, `elems->resetMaterialStatuses()` discards the failed trial state.

22.3 Prescribed Degrees of Freedom Are Balanced by Reactions

The residual on directly prescribed degrees of freedom is not used as an unknown equation in the reduced system. Instead, the external force on those degrees of freedom is set to the value needed to balance internal, damping, and inertial forces. This makes the reaction force an output quantity rather than an unknown correction.

22.4 Matrix Update Frequency Changes the Nonlinear Method

If the matrix is rebuilt every nonlinear iteration with a tangent stiffness, the algorithm behaves most like Newton's method. If the matrix is rebuilt less often or uses a secant/elastic matrix, the method trades convergence rate for cheaper iterations and sometimes improved robustness.

22.5 Linear Solver Accuracy Can Affect Nonlinear Acceptance

For iterative linear solvers, the correction is approximate. Adaptive linear tolerance allows early nonlinear iterations to be solved loosely, but the code can require a tight final correction before accepting the nonlinear step. This is especially relevant for deflated FGMRES and AMG preconditioned paths.

23 Compact Algorithm Reference

```
Model::solve:
  export initial data
  while not solver.terminated:
    solver.solveStep()
    export accepted step

Solver::solveStep:
  runBeforeEachStep()
  solve()
  runAfterEachStep()

SteadyStateNonLinearSolver::solve, direct control:
  converged = false
  while not converged:
    load = loads(time)
    trial_r = prescribed_boundary_values(trial_r, time)
    updateFieldVariables() with zero ddr
    computeForcesAtIntegrationTime(frozen=true)

    it = 0
    while not converged and it < maxIt:
      if matrix_update_required:
        K = assemble_structural_matrix(stiffness_type)
        Keff = constraint_transform(K)
        factorize_or_setup_linear_solver(Keff)

        f = reduce(residuals)
        ddr = solve(Keff, f)
        full_ddr = expand_and_apply_dependencies(ddr)
        trial_r += full_ddr

        computeForcesAtIntegrationTime(frozen=false)
        evaluateErrors()
        converged = errors_below_tolerances and it+1 >= minIt
        export iteration
        it += 1

    if converged:
      fully_converged = true
      computeForcesAtStepEnd(frozen=false)
    else if dt can be reduced:
      restore committed state
      reset material statuses
      reduce dt and retry
    else if limit tolerances are satisfied:
      fully_converged = false
      computeForcesAtStepEnd(frozen=false)
      converged = true
```



```

        else:
            terminated = true
            return

        adapt next dt

Solver::runAfterEachStep:
    compute energies
    r = trial_r
    f_int_old = f_int
    f_ext_old = f_ext
    commit material statuses
    update termination and next dt

```

24 Reading Solver Output

The nonlinear solver prints a table per step. The key columns map directly to the internal quantities:

Table 14: Typical nonlinear output fields.

Output field	Meaning
<code>time</code>	Current target time/load coordinate.
<code>timestep</code>	Current Δt .
<code>iter</code>	Nonlinear iteration number.
<code>residuals</code>	<code>resErr</code> , the normalized force residual error.
<code>increments</code>	<code>disErr</code> , the normalized correction/increment error.
<code>energies</code>	<code>eneErr</code> , the normalized residual-work error.

For iterative linear solvers, the more detailed event file records each linear system inside each nonlinear iteration. This is the right place to check whether more nonlinear work is caused by changed matrix updates, worse preconditioner quality, loose linear tolerance, or increased linear iterations.

25 Summary

The OAS nonlinear solver is a load/time-step Newton-like equilibrium solver with:

- a clear separation between committed state `r` and trial state `trial_r`;
- full-space force assembly followed by reduced-space linear solves;
- explicit constraint and prescribed-boundary handling through `NodeContainer`;
- residual, increment, and energy convergence checks separated by physical field;
- restart and adaptive time-step control when convergence is difficult;
- optional relaxed-tolerance acceptance;
- optional indirect displacement control using two linear solves per nonlinear iteration;
- detailed runtime and linear-solver profiling hooks.

The central nonlinear equation is the active equilibrium residual

$$R_f(r^{\text{tr}}, t) = 0,$$

solved by repeated linear corrections

$$K_{\text{eff}} \Delta r_f = R_f,$$

where the definitions of K_{eff} , R_f , and the convergence scales are determined by the current stiffness update policy, constraints, physical fields, and selected linear solver backend.